# Project - Low Level Design

# on

# CI/CD for Node.js Retail App

*Course Name: DevOps*

***Institution Name:*** *Medicaps University – Datagami Skill Based Course*

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1 | Amit Patidar | EN22CS301114 |
| 2 | Aniket Kushwah | EN22CS301124 |
| 3 | Anuj Singh Rathore | EN22CS301166 |
| 4 | Anurag Didolkar | EN22CS301169 |
| 5 | Arsh Patidar | EN22CS301204 |
| 6 | Avani Gupta | EN22CS301236 |

*Group Name: Group 09D2*

*Project Number:* DO-21

Industry Mentor Name:

University Mentor Name:

Academic Year: 2025–2026

# Table of Contents


1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. References

# 1. Introduction

## 1.1 Scope of the Document

This document presents a detailed Low-Level Design (LLD) for implementing a CI/CD pipeline for a Node.js-based Retail Application. It covers all technical aspects including system components, workflows, automation processes, containerization, and deployment strategies.

The scope includes:

- Code integration and version control
- Automated build and testing pipeline
- Containerization using Docker
- Deployment using Kubernetes
- Internal component interactions and configurations

This document ensures that developers and DevOps engineers can understand and implement the system efficiently with minimal ambiguity.

## 1.2 Intended Audience

*This document is intended for:*

- ***Developers:*** *To understand application structure and integration with CI/CD*
- ***DevOps Engineers:*** *To implement and manage automation pipelines*
- ***System Architects:*** *To review system design and scalability*
- ***Test Engineers:*** *To validate automated testing workflows*

- **Project Evaluators:** To assess technical implementation and design clarity

## 1.3 System Overview

The system is designed to automate the end-to-end software delivery lifecycle of a retail web application. It integrates multiple DevOps tools to ensure seamless and efficient deployment.

**Key Functionalities:**

- Automatic triggering of pipeline on code changes
- Continuous integration with build and test automation
- Containerization of application using Docker
- Continuous deployment to Kubernetes cluster
- Monitoring and scalability support

**Workflow Summary:**

Whenever a developer pushes code:

1. The version control system detects changes
2. CI pipeline is triggered automatically
3. Application is built and tested
4. Docker image is created and stored
5. Kubernetes deploys the updated application

This system reduces manual effort, improves reliability, and speeds up delivery cycles

# 2. System Design

## 2.1 Application Design

The application follows a **modular and layered architecture**:

### 1. Presentation Layer (Optional)

- Provides user interface (UI)
- Can be built using HTML/CSS/JavaScript or React

### 2. Application Layer (Backend)

- Built using Node.js and Express
- Handles routing and business logic

### 3. Service Layer

- Processes business operations like product handling, order management

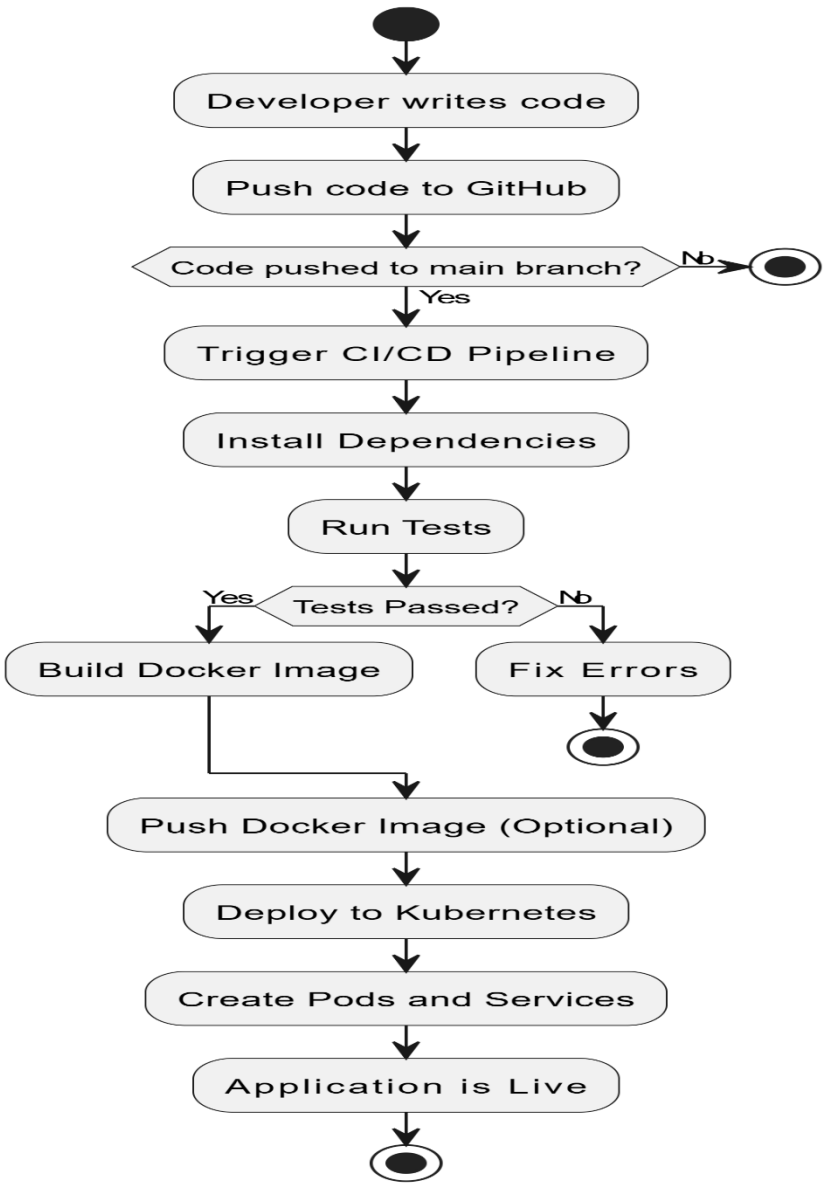
### 4. Deployment Layer

- Docker containerizes the application
- Kubernetes orchestrates deployment

**Project Directory Structure:**

```
retail-app/
├── src/
│   ├── routes/
│   ├── controllers/
│   ├── services/
│   ├── models/
│
├── app.js
├── package.json
├── Dockerfile
├── .github/workflows/ci.yml
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
```

## 2.2 Process Flow




The system follows a **fully automated CI/CD pipeline flow**:

**Step-by-Step Flow:**

1. Developer writes or updates code

2. Code is committed and pushed to GitHub repository

3. GitHub Actions/Jenkins detects the change

4. CI pipeline starts execution
5. Dependencies are installed automatically
6. Code is compiled and tested
7. Docker image is built from Dockerfile
8. Image is tagged and optionally pushed to Docker Hub
9. Kubernetes pulls the latest image
10. Deployment is updated with new version
11. Application becomes accessible to users

**Pipeline Stages**

- Source Stage
- Build Stage
- Test Stage
- Containerization Stage
- Deployment Stage

## 2.3 Information Flow

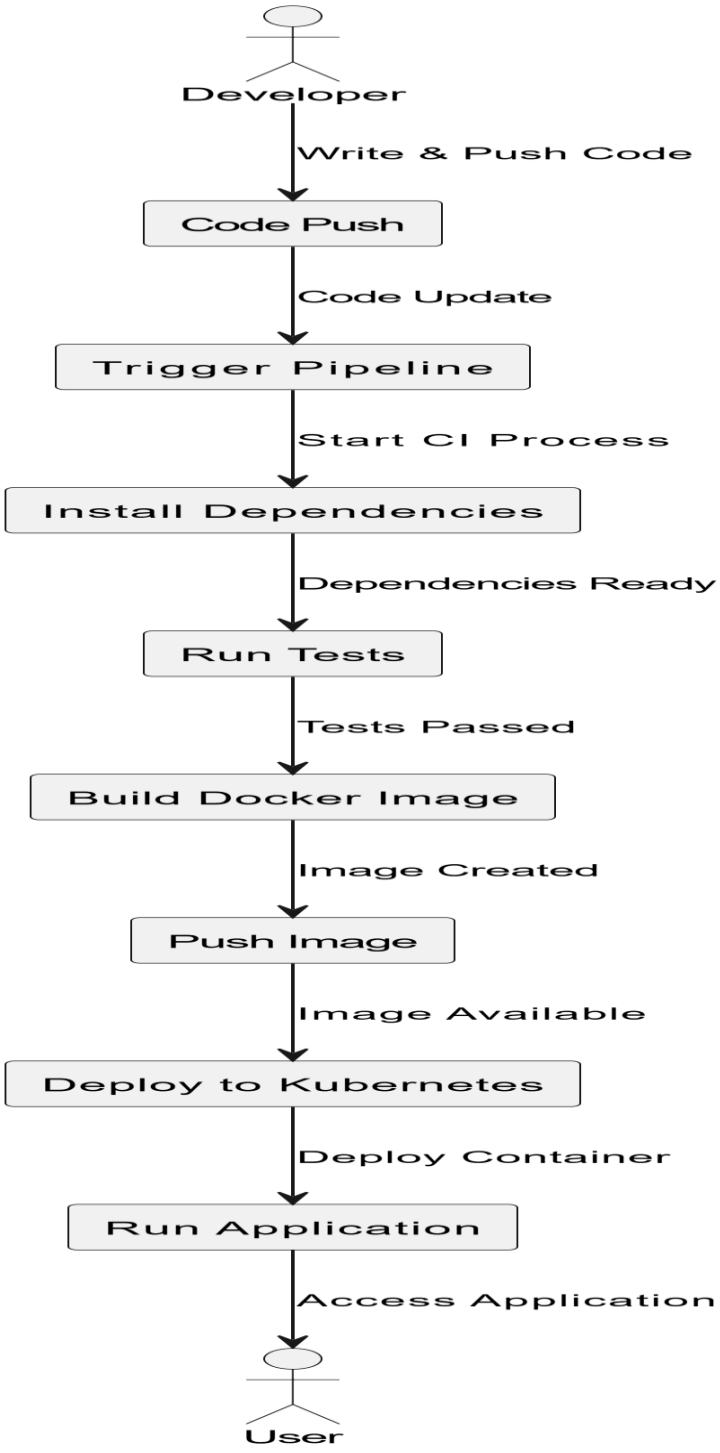
The system ensures smooth and structured data flow across components.

Flow Description:

- Developer sends code → GitHub repository
- GitHub triggers CI/CD tool
- CI/CD tool executes build/test scripts
- Docker builds container image
- Image is transferred to Kubernetes
- Kubernetes runs container and exposes service

**Data Flow Representation:**

Developer → GitHub → CI Tool → Docker → Kubernetes → End User


Developer
Write & Push Code
Code Push
Code Update
Trigger Pipeline
Start CI Process
Install Dependencies
Dependencies Ready
Run Tests
Tests Passed
Build Docker Image
Image Created
Push Image
Image Available
Deploy to Kubernetes
Deploy Container
Run Application
Access Application
User

## 2.4 Components Design

1. Version Control System (GitHub)

- Maintains source code repository
- Tracks changes using commits
- Enables collaboration
- Triggers CI/CD pipeline via webhooks

## 2. CI/CD Tool (GitHub Actions / Jenkins)

Responsibilities:

- Automates build process
- Executes test cases
- Integrates code changes
- Builds Docker images

Key Features:

- Workflow-based execution
- Event-driven triggers
- Parallel job execution

## 3. Docker

**Responsibilities:**

- Packages application with dependencies
- Creates lightweight, portable containers

Key Components:

- Dockerfile
- Docker Image
- Docker Container

## 4. Kubernetes

Responsibilities:

- Deploys containerized application
- Manages scaling and load balancing
- Ensures high availability

Key Objects:

- Deployment
- Pod
- Service

## 5. Node.js Application

Responsibilities:

- Handles API requests
- Processes business logic
- Sends responses

Key Features:

- Lightweight and fast
- Event-driven architecture

## 2.5 Key Design Considerations

*1. Scalability*

- *Kubernetes enables horizontal scaling using replicas*
- *System can handle increased user load*

*2. Reliability*

- *Automated pipelines reduce human errors*
- *Kubernetes ensures self-healing of failed containers*

*3. Consistency*

- *Docker ensures same environment across all stages*

- Eliminates environment-related issues

**4. Security**

- Secure access using authentication tokens
- Environment variables for sensitive data
- Controlled access to repositories

**5. Maintainability**

- Modular project structure
- Easy debugging and updates

**6. Performance Optimization**

- Fast build and deployment cycles
- Efficient resource utilization using containers

## 2.6 API Catalogue :

**1.GET /**

- Description: Health check endpoint

- Response:

{
"message": "Retail App Running"
}

2. GET /products

- Description: Retrieve list of products
- Response:

[
{ "id": 1, "name": "Product A", "price": 100 },
{ "id": 2, "name": "Product B", "price": 200 }
]

3. POST /order

- Description: Create a new order
- Request:

{
"productId": 1,

```
  "quantity": 2
}
```

Response:

```
{
  "message": "Order placed successfully"
}
```

4. GET /orders (Optional)

- Description: Fetch all orders
- Response: List of orders

## 3. References:

- Node.js Official Documentation
- GitHub Actions Documentation
- Docker Official Documentation
- Kubernetes Official Documentation
- DevOps Best Practices Guides